4

Chapter 4 – Community driven development

Open source software is often developed by a group of people who normally have no governing authority or organizational support. What are the strategies and methods they follow? Which tools do they use at the various phases of the software development life cycle? Are these tools available for free? This chapter provides the answers to all of the above questions.

In this chapter you will learn about:

- Various tools used in an open source software development project.
- Processes of designing and writing the programs.
- Managing different versions of the source code and creating new builds.
- Methods of testing and issues tracking.
- How open source software is released.

4.1 Community driven development: The big picture

A ***community*** is a diverse group of people sharing their thoughts, ideas, work, and experiences, at a common place. The term “community” as used in this chapter is not an exception to this definition. In the software industry, ***community driven development*** broadly means an initiative under which a group of technologists work together with a common vision of producing open source software.

Most of the OSS development communities who are successful today, organize themselves in a similar way as a professional organization or proprietary software company. This is why in a well structured OSS community; you can find each member has a job role within the team. Groups of people performing similar jobs, form sub teams within the community. *Figure 4.1* gives you an overview of such a community, where multiple sub teams collaborate to develop open source software.

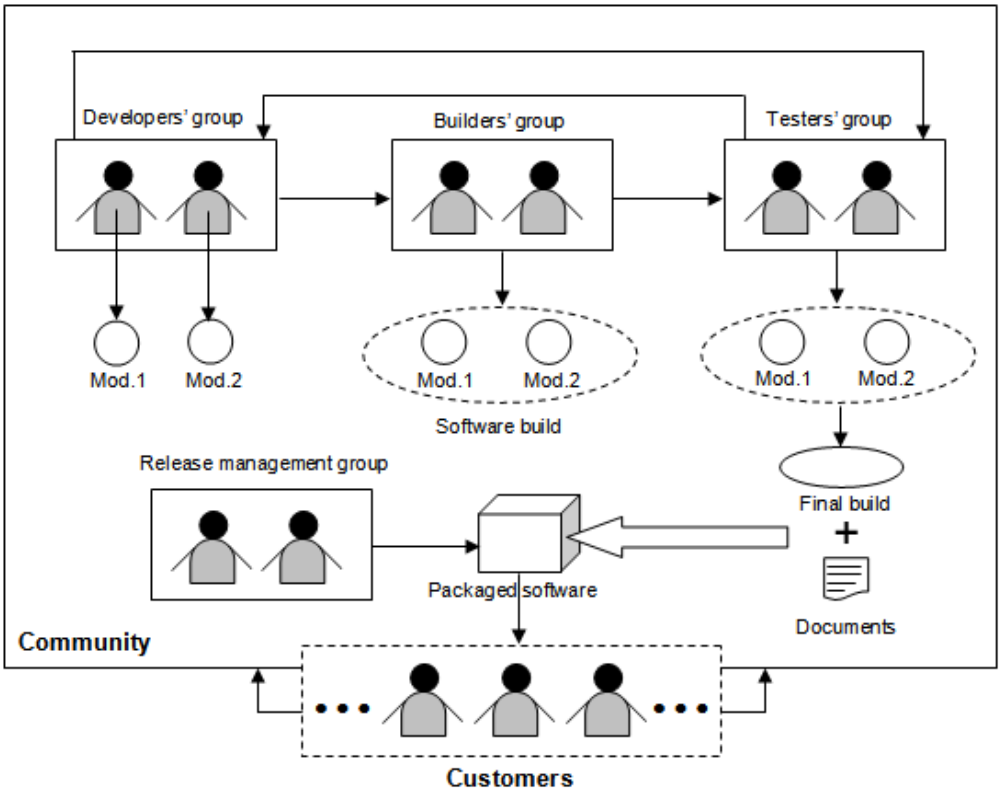


Figure 4.1 – Community driven software development

As shown in the figure, the ***developers' group*** is responsible for writing code for different independent modules of the software. The ***builders' group*** takes these modules from the developers and put them together to build a new version of the software. These software builds are internally tested by the ***testers' group***. In case a bug or failure is found, they report back to the appropriate developer who debugs the code and includes a proper fix. This is an iterative process that repeats itself until the product satisfies all its requirements and becomes error free to the maximum possible extent. Once the objectives of the project are met, the ***release management group*** packs together the final version (final build) of the software with all the necessary documents, and then hands it over to the customers.

OSS development communities, especially those which are self driven, offer their team members the flexibility of shuffling their job roles. This means that a developer may perform the role of a tester, a tester may work as a build team member, and so on. Most importantly, these communities always keep the door open for the users (customers), so that they can also contribute effectively into the project and become part of the community.

Software development tools are a collection of programs that help with specific tasks during the software development life cycle. They are normally used for project management, and to perform repetitive, and time consuming tasks. There exist both, proprietary and open source development tools. For an open source project, it should be

obvious that the latter one is the preferred choice. There are also a few instances, where communities use their self developed tools.

The next sections describe the various processes and tools used by the OSS development communities in detail.

4.1.1 Developers' group: Software design and development

You can think of ***software design*** as the blueprint that defines the architecture and behavior of a product, and ***coding*** as the process of implementing the design.

A community driven software development typically starts as a project to address problems generic across the society. These problems are technically termed ***pain points***, and include a list of requirements that the software is expected to fulfill. The designers' group, which in many cases is also part of the developers' group, takes these requirements as input and writes algorithms which are cost effective and optimal in terms of performance.

Software designers should be the most experienced and knowledgeable members of the community as their job requires thorough understanding of the entire system. They also must be aware of every alternative to overcome all the probable barriers of implementing the design. Following are the two most important points that a designer must analyze closely:

- **Hardware platform:** The designer must know the capabilities of the hardware, with which the software is going to interact. Resources like number and speed of the processors, I/O device response time, available system cache, primary memory and secondary storage space etc. are the most important concerns.
- **Operating system:** The operating system (OS) provides the environment for running applications. Process and thread handling mechanisms vary from one OS to another. Moreover, the algorithms of job scheduling, CPU time allocation, avoiding deadlock situations are also different. The designer must take these factors into account.

A perfect design should be self explanatory and easy to understand. All the algorithms and flowcharts should include detailed explanations and other necessary documentations including time-space complexity and the expected behaviors of the system under best and worst scenarios.

Unified Modeling Language (UML) is a widely accepted standard used for software design. It allows a designer to describe the structure, behavior, and interactions among different sub units of the system with the help of various simple diagrams.

Following design, the software development life cycle enters the coding phase. At this stage, the design is first sub divided into multiple smaller units. Different development groups code these sub units separately as individual modules.

Coding is the process of translating the steps of an algorithm, into multiple computer instructions, with the help of a programming language. Choosing the proper language for writing the source code is critical. A community should always encourage using languages

which have gained industry wide trust and with which the development team is well versed. Some communities prefer using interpreted languages, since they consume less time to create the executable modules as compared to the compiled languages.

Automatic code generators are often helpful in developing big and complex code snippets. *iCodeGenerator* is an open source code generator offered by SourceForge (http://sourceforge.net/projects/codegenerator/).

Transition between the design and code generation phases in a community driven development typically tend to overlap with each other in a project. Developers sometimes start coding parts of a system, while other parts are yet to be designed. Although, this kind of approach of running two phases in parallel saves time; it may result in duplication of effort in the event of a change in design.

4.1.1.1 Version control

Software development is an incremental process where functionality is added or removed one by one. At any given time the code of different modules may undergo frequent modifications by different individuals or groups which create different versions of the same code. ***Version control*** (or ***revision control***) is the mechanism to manage all of these versions in an effective way.

There are two varieties of version control systems:

- **Centralized Version Control System (CVCS)** – This is perhaps the most widely used version control system to-date. It works in a centralized manner which means that developers and testers around the world have to connect to a server, where a central repository for the entire project is stored.

  Concurrent Versions System (CVS) is a very well known open source CVCS (http://ftp.gnu.org/non-gnu/cvs/). To get a private copy of a particular file, you first need to check it out from the repository. Once done with your changes, you may commit or save your work into the repository by checking the file in. CVS will automatically increment the version (or revision) number by 1 and record all the necessary information about the change in its log. If someone else had already committed some other changes to the same file, CVS will automatically merge your changes with those of the other person. This system also allows a user to track back the old versions of a file. By comparing two successive versions of a file, you can easily determine modifications introduced.

  One of the main drawbacks of CVCS is that developers may interfere with each other's working environment. For example, a small bug injected through a code change, becomes a big issue when it causes various other modules of the entire code base to fail.

- **Decentralized / Distributed Version Control System (DVCS)** – This type of version control allows a developer or tester to create their own code branch, that is, they can maintain versions of the code in a decentralized manner. A DVCS provides all the functionalities of CVCS, plus the flexibility of creating a local

repository. Git, Bazaar, Monotone, Mercurial and Darcs are a few examples of open source DVCS software.

Did you know?

During the early days of Linux® kernel development, Linus Torvalds used CVS for version management. In 2002, Linus switched to BitKeeper; however, since BitKeeper became proprietary in 2005, Linus started writing a completely new DVCS, called git. The official web site of git (http://git-scm.com/) describes it as – "*... a free & open source, distributed version control system designed to handle everything from small to very large projects with speed and efficiency. Every Git clone is a full-fledged repository with complete history and full revision tracking capabilities, not dependent on network access or a central server. Branching and merging are fast and easy to do.*"

4.1.2 Builders' group: Software building

Programs are normally written in smaller units that are compiled before they can be executed. The resulting object modules are then bound together in a process known as ***linking*** to form a complete software system. ***Software building*** refers to the sum of ***compiling*** and ***linking*** all the source code of a project.

Software building is a dynamic process: People constantly add, delete and modify the source code files under the project's central repository. To make these changes effective, the modified code needs to be recompiled and linked regularly. Development communities generally have a separate build team to carry out this task. It is important to mention that the build process recompiles only those source code files which were modified since the previous build was released. For the rest of the elements which remained unchanged, the older compiled modules are picked up. *Figure 4.3* illustrates this fact.

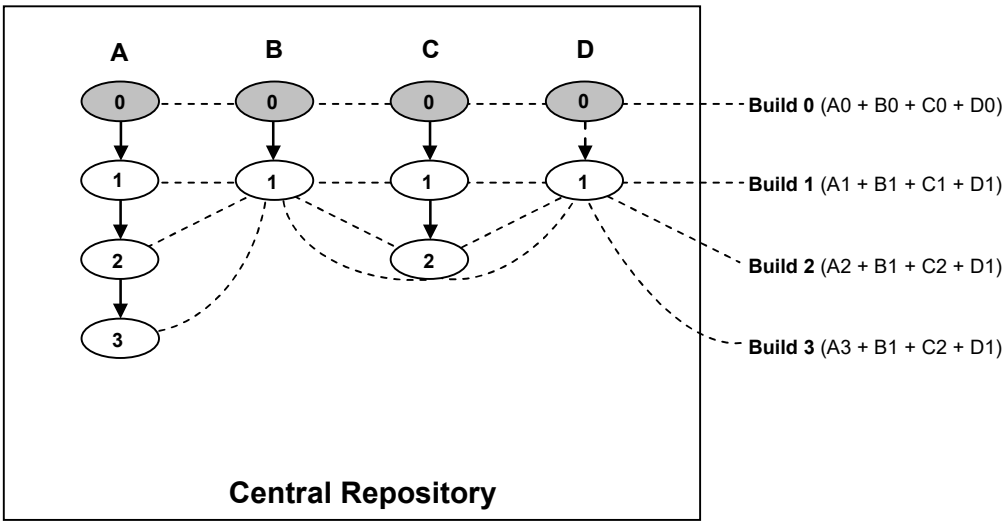


Figure 4.3 – Software builds

In the figure, the central repository contains four source code files A, B, C and D. The base version of these files is illustrated with the number zero, so A0 represents the base version of source code file A for example. When compiled and linked they create 'Build 0'. On the next iteration, all the files were modified and their versions were upgraded to 1. Since all the files were modified, all are compiled and linked to create 'Build 1'. On the net iteration, only source code file A and C are modified creating second versions of them (A2 and C2). As a result when creating 'Build 2', only A2 and C2 are compiled, and then linked with B1 and D1. Similarly, 'Build 3' is created based on the newly compiled A3 and precompiled B1, C2 and D1.

Berczuk and ***Appleton*** have divided software builds into the following three distinct categories in their book *"Software Configuration Management Patterns"* [2], and an additional paper with ***Konieczka***, *"Build Management for an Agile Team"* [3]:

- **Private build** – A private build is a new version of a developer's personal branch in a DVCS. Naturally, this type of build is only visible to its owner and he can access it locally even when working offline.
- **Integration build** – An integration build is created from the compilation and linking of the main project branch and is available to all within the community for use.
- **Release build** – A release build is the final version of the software to be shipped. It is basically a snapshot of a stable integration build which users use as a product.

Frequent builds allow the source code to undergo less number of modifications. This reduces the effort needed for debugging and fixing the new errors. On the other hand, the build process involves a lot of repetitive tasks which can be automated by means of open source tools like *make*, *ant*, *maven*, and *scons*.

So far we have discussed about the building process in a project environment. However, since the code is open, at anytime users can tailor it at their own premises and get customized versions of the product. A later section discusses how you can rebuild and install open source software after changing its source code.

4.1.3 Testers' group: Software Testing

Software testing as defined by ***Glenford J. Myers*** in his book *"The Art of Software Testing"* [4], is *"the process of executing a program with the intent of finding errors"*.

Conventionally, there are two varieties of software testing – ***black box*** and ***white box***.

- **Black box testing:** In this type of testing, the actual output of the code is compared with the expected result. Testers are supposed to feed the system with a set of inputs, and note down the corresponding outputs. In case of any mismatch between the obtained and expected results they report it to the developer without analyzing the code.
- **White box testing:** In contrast to black box, in this type of testing, testers look into the source code of the product and carry out some investigation in the event of

failures. This requires a developer to put less effort and time for analyzing the error before fixing it.

With open source software, every person, no matter whether he is a member of the community or not, gets equal opportunity of performing white box testing as the source code is accessible to all.

Testing also requires planning and a proper infrastructure. At different stages of development, the code undergoes various levels of testing:

- ***Unit testing:*** This level is used to test individually each of the units that make up a software
- ***Integration testing:*** In this level, all the stable modules are integrated with each other to form the entire system, and tested as a whole.
- ***System testing:*** This testing ensures the software is operational and satisfying all its requirements.
- ***Alpha testing:*** In this stage, the software is given to the users internal to the community, to judge its performance against practical deployment.
- ***Beta testing***: After fixing the errors discovered during alpha testing, the community releases the software to the external users as a beta version, with a disclaimer that it may fail in unplanned use cases. The community expects external users to provide feedback about these unplanned scenarios.

Small scale open source software without any organizational governance may not undergo formal testing. However, this software tends to be quickly adopted because it provides a solution to a common problem. When a bug is encountered, the user will normally poke through the code to make things work again, or he can customize it to his own needs. This is the way in which many people become members of OSS communities. When the software becomes so useful and with enough followers, some users take the software to the next level by enhancing it, and formally releasing it to the community.

OSS development often spends less time and budget to test their product. Rather, they leverage the community to test and fix the code themselves; and to provide feedback for new functionality. In addition, this unique approach makes open source software very robust.

4.1.4 Release management group: Packaging

Users give preference to a product that is well packaged. In an open source software, the core item of a package is the source code. A standard package includes accessories like an installation guide, a user manual, and other information. Packages of software written in high level languages like C or C++ that require compilation of the source code, optionally may contain a pre-compiled executable version of the product. All of these elements are put together within a single directory and compressed into a single file.

The format of the compressed file depends on the operating system on which the software is meant to run. On Linux or UNIX®, packages are presented in formats like ***.tar, .gz*** etc.

which are created using compression utilities like ***tar, gzip, bgzip, bzip2***. On Windows®, the distribution package is ***zip***. If the software has operating system specific versions, each of them is released in separate packages.

Once the package is prepared, it goes through a small test by a few developers to ensure the final version of the product can be decompressed, installed, and works correctly.

Then it is published in one or more internet Web sites for download. In some cases, if there is budget, CDs or DVDs can be ordered online at no charge.

4.1.5 Release management group: Releasing

Releasing software means making a completely new or upgraded version of a software available to the users. A new release introduces a product into the market and an upgraded version provides fixes for errors found in a previous released version and optionally adds new features.

To distinguish the versions from each other, each OSS community follows its own convention of using unique numbers for every release. Provided below is a template of such numbering scheme for the file names of the compressed package. The file extension is not shown, but it would be ***.tar, .gz, .zip*** and so on as discussed in the previous section:

```
Abcdefg x.y.z < Alpha | Beta >
```

In the above example, `Abcdefg` is the name of the software and `x, y,` and `z` represent the ***major, minor*** and ***micro*** components of the unique release number respectively. A release containing only small fixes for a previous version, is considered as a small or minor release and thus associated with the same release number as that of the previous one, incrementing its last digit (micro component) by 1. For example, if a product was released as `Abcdefg 2.0.5`, a small release would be represented as `Abcdefg 2.0.6`. A critical bug fix is implied by increasing the minor component and resetting the micro component which makes the new release to be identified as `Abcdefg 2.1.0`. If new functionalities are added, the version number would be upgraded to `Abcdefg 3.0.0`. When the software with limited functionality is made available to internal users for testing, this interim version is called Alpha as in `Abcdefg 3.0.0 Alpha`. An alpha version helps the development community collect error reports form the users. Depending on the count and sensitivity of the errors discovered, communities may also decide to release multiple alpha versions like `Abcdefg 3.0.0 Alpha 1`, `Abcdefg 3.0.0 Alpha 2`, and so on. When the alpha round is over, the feature complete product is given to third parties outside the community. This is called a `Beta` release. Following the example, the release would be called `Abcdefg 3.0.0 Beta`. With sufficient testing and debugging through alpha and beta cycles, the software finally becomes ready to be officially shipped. At this conclusive stage, the community announces ***"General Availability (GA)"*** of their product.

After a successful release, the developers' group work is divided in two: The first one is to maintain & support the already released version, and the second one is to further improve the product with new functionalities to be included in the next version. At this point, the central repository of the version control system is branched into two. The branch where

maintenance works will occur is called the ***release branch***. Stable snapshots of this branch are released as minor versions of the product at the same major line (for example, `Abcdefg 3.1.0`, `Abcdefg 3.2.0` and so on). Fixes for high potential failures which cannot wait until next minor release are distributed to the customers via a sub branch of the current minor release line such as `Abcdefg 3.2.1`, `Abcdefg 3.2.2`, etc.

The branch where developers work on the new version (`Abcdefg 4.0.0` for this example) is called the ***mainline development branch.*** This branch progresses in parallel with the release branch.

A community should continue to provide fixes for previous versions until the majority of the existing users start using the latest version of the software.

4.2 Installation and issue tracking

After an OSS is released and is available to users, users have to install the product. In contrast to proprietary software, OSS may require users to compile the source code, and perform some configuration before the installation. This is described in this section. The section also describes how users can provide feedback to the community using issue tracking.

4.2.1 Installation

Before a user can install the software in his system, he needs to extract it by decompressing the package. Many users often attempt to install the software without reading the instructions. It is recommended to at least review the `README` or `INSTALL` files. Typical installation steps for open source software are:

- Understanding the system configuration
- Compiling the source code
- Executing the installation file.

For example, on a Linux or UNIX based system, the sequence of commands below accomplish these three steps:

```
# ./configure

# make

# make install
```

The first command is executed from the location where the software package is extracted. It invokes a tiny program called `configure` (generally included with the package) to detect the hardware and software specifications of the machine and create a `Makefile` containing the set of instructions to complete the installation process.

The second command, (**`make`**) compiles the source code of the software as dictated by the `Makefile`.

The third command (**`make install`**) reads the configuration specifications of the computer from the `Makefile` and installs the software into it. Although the first two commands can be executed by any normal user, the last one needs root access.

If the OSS package does not contain the `configure` script, users are provided with a `Makefile` they can edit with their computer's configuration and start the setup process from the second step.

Note that users can change the source code at any time and go through this process again to install the modified version of the product.

4.2.2 Issue tracking

Although the term "issue" may be interpreted as a bug or problem in the software development world, in reality it is more than that. An issue may refer to a bug, but also to a change request, where the change is related to a piece of existing code, design, feature or even software documentation.

OSS projects typically use free issue tracking tools like Abuky, Bugzilla, Buggit, BugRat, GNATS, JitterBug, and Mantis. An issue can be discovered by a developer, a tester or even a user. Once identified it is reported to the community by logging a record in the ***issue tracking database***, managed by the tool, with all the necessary details. Every single issue record is associated with a unique number for identification purpose. *Figure 4.2* shows different states of an issue throughout its entire life cycle.

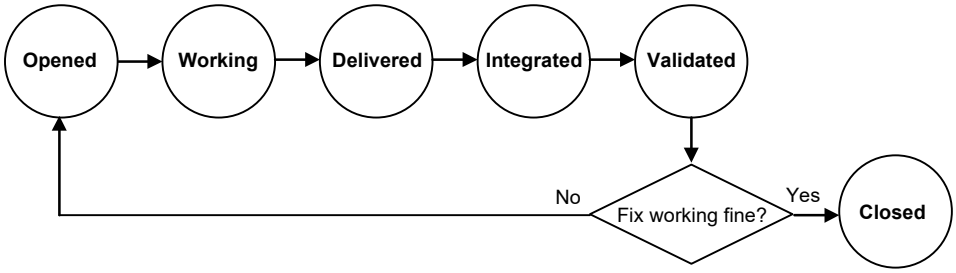


Figure 4.2 – Life cycle of an issue.

When a person ("the originator") finds a problem, he logs a defect in the issue tracking database, and the state of the issue is set to ***opened***. Thereafter, someone from the development team tries to reproduce the error and takes ownership of the issue if he validates it is indeed a bug. At that point the state of the issue is updated to ***working***. Once the developer (owner) fixes the problem, he commits the changes required into the central repository, and updates the state to ***delivered***. When these changes are included in the build of the project branch, the state changes to ***integrated***. If the build passes all the required tests, it is made available to the entire community and the issue state changes to

validated. The originator can now verify that the new build resolves the problem. If the error reoccurs, the state is switched back to ***opened*** and the whole cycle starts again; otherwise, the issue is ***closed***.

There are variations to the life cycle illustrated in *Figure 4.2*. It is common to find new ***opened*** issues being ***closed*** very soon due to these reasons:

- **Duplicate:** If someone opens an issue for an error which is already known to the development team, the issue is closed as a ***duplicate***.
- **Non-reproducible**: When an issue cannot be recreated under a similar environment, the issue is closed as ***non-reproducible***.

Such instances are very common as users not following an OSS project closely are also able to report failures. Moreover, there are instances where users open issues without understanding a feature properly. This can cause the issue tracking database of a popular OSS to grow considerably with invalid issues. To avoid this situation, it is the responsibility of the community's active members to follow up the issues as soon as they are opened by either accepting or declining them with proper justifications. On the hand, users should analyze a problem before opening an issue. They should at least perform some basic research by discussing the problem in blogs or discussion forums, or searching in the issue tracking database. Another option is to e-mail the development team directly, and only open an issue when they confirm it is a new problem.

4.3 Exercises

1. Visit Mozilla's bug tracking system at https://bugzilla.mozilla.org/ and see how an open source community tracks issues. Mozilla accepts bug reports and feature enhancement requests for all their products through this portal.
2. Know more about open source software testing from http://www.opensourcetesting.org/.
3. Learn about "Linux Software Packaging History and Concepts" from http://wiki.rpath.com/wiki/Appliance_Development:Linux_Software_Packaging_History_and_Concepts.
4. The Apache community uses Subversion (SVN) revision control system and allows everyone to access the source code over the internet. Know how you can get inside their code repository and work with the contents from http://www.apache.org/dev/version-control.html#source-code-repositories.

4.4 Summary

In this chapter, you learned how open source software is developed as a community initiative and how a project goes from an idea to a featured product. The chapter discussed all the stages of the software development life cycle, such as designing, coding, testing, building, packaging and releasing; with focus on the different groups who participate at each stage, and also on the open source software tools used.

4.5 Review questions

1. What does the term “community driven development” mean in the software industry?
2. State the key difference between CVCS and DVCS.
3. Name some open source issue tracking tools.
4. What are the items typically included within an OSS package?
5. What are the steps to install open source software in a computer?
6. Which of the following is a valid format of a Windows specific OSS package?

   A. .tar

   B. .zip

   C. .gz

   D. A and C

   E. All of the above.
7. Which of the following is used during the design process of an OSS?

   A. UML

   B. XML

   C. CVS

   D. Git

   E. None of the above
8. *myproduct 3.4.5* – in this scheme of version numbering, which is the “minor” component?

   A. myproduct

   B. 3

   C. 4

   D. 5

   E. None of the above
9. Which two of the following tools can be used to automate software build process?

   A. Make

   B. Apache

   C. Ant

   D. Darcs

   E. None of the above

10. Which of the following is not a state in the life cycle of an issue?

    A. Opened

    B. Fixed

    C. Closed

    D. Integrated

    E. Validated